Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Bir algoritmanın sağlaması gereken şartlar: doğru, basit, genel, kolay gerçeklenebilir ve verimli (efficient) olmak.
- Verimlilik iki kaynakla ölçülür: zaman (time) ve yer (space).
- Çalışma süresinin giriş boyutu n ile nasıl büyüdüğü: $\log n$, n, $n \log n$, n^2, 2^n, $n!$ karşılaştırması.
- Çalışma süresini etkileyen faktörler: donanım, derleyici (compiler), programlama dili ve veri.
- Adım sayma ile karmaşıklık hesabı, temel işlem (basic operation) kavramı.
- Asimptotik analiz (asymptotic analysis) ve asimptotik notasyon: Big O, Big Theta, Big Omega.
- En iyi durum (best case), en kötü durum (worst case) ve ortalama durum (average case).
- Sırasız dizide doğrusal arama (sequential search) analizi.
- Insertion sort algoritması ve analizi.
- Karmaşıklık sınıfları ve sık kullanılan toplam formülleri.

Hocanın Özellikle Vurguladığı Kısımlar

- Algoritmaya doğrudan kodla başlamayın. Önce akış diyagramı veya yarı kod (pseudo code) ile tasarlayın. Koddaki hata düzeltmeleri "yama" gibi olur ve algoritmayı gereksiz büyütür.
- Çalışma süresini saniye ile ölçmek iyi bir yöntem değil. Süre donanıma, derleyiciye ve dile bağlıdır. Algoritma sistemden bağımsız değerlendirilmeli.
- Karmaşıklık hesabında sabitler, katsayılar ve küçük terimler önemsizdir. Önemli olan n'e bağlı en büyük terimdir.
- Bu dönem yalnız Big O kullanılacak. Theta ve Omega algoritma analizi dersinde (3. sınıf) işlenecek.
- Bu dönem hep worst case üzerinden gidilecek. Average case yalnız bir kez, kavram anlaşılsın diye gösterildi.
- Karmaşıklık sınıflarını ezberlemeye gerek yok. Sabit iş $O(1)$, her adımda ikiye bölünüyorsa $O(\log n)$; diğerleri döngülerden çıkarılır.

Kısa Tekrar Notları

- İyi algoritma: doğru, sonlu, basit, genel, kolay gerçeklenen, zaman ve yeri verimli kullanan.
- Temel işlem: n'e bağlı olarak en çok tekrar eden iş.
- Sadeleştirme sırası: sabit terimi at, katsayıyı at, en baskın terimi al. $4n^2+2n+9 \Rightarrow O(n^2)$.
- Big O = asimptotik üst sınır (upper bound), Theta = sıkı sınır (tight bound), Omega = alt sınır (lower bound).
- $m \times n$ matriste m küçük ve sınırlıysa $O(n)$ denebilir. m de n gibi büyüyorsa $O(mn)$ veya $O(n^2)$ denmeli.
- Best/worst/average farkı, adım sayısı belli olmayan (while ile, bir koşul sağlanınca çıkan) algoritmalarda ortaya çıkar.
- Dizinin en büyük elemanını bulmak: best = worst = $O(n)$.

- Sırasız dizide arama: best $O(1)$, worst $O(n)$, average $\frac{n+1}{2} \Rightarrow O(n)$.
- Insertion sort: best $O(n)$ (sıralı dizi), worst $O(n^2)$ (ters sıralı dizi).
- Quicksort: average $O(n \log n)$, worst $O(n^2)$; yine de en çok kullanılan sıralama algoritması.

Detaylı Açıklamalar

Algoritmanın Sağlaması Gereken Şartlar

- **Doğru (correct):** Her geçerli giriş için doğru sonucu verir. Sonlu sayıda adımda biter, sonsuz döngüye girmez.
- **Basit (simple):** Hoca bunu Occam'ın usturası (Occam's razor) ile bağladı. Aynı problemi çözen en basit yol tercih edilir.
- **Genel (general):** Belirli birkaç sayı için değil, problemin genel hâli için çalışır.
- **Kolay gerçeklenebilir (easy to implement):** Tasarımda basit görünen şey kodda içinden çıkılmaz hâle geliyorsa tasarım kötüdür. Daha basit bir çözüme indirgemenin yolu aranmalı.
- **Verimli (efficient):** İlk dört şart sağlandıktan sonra bakılır. Zamanı ve yeri ne kadar iyi kullandığı ölçülür.

Büyüme Hızlarının Karşılaştırılması

Hoca, her işlemin yaklaşık 1 ns sürdüğü bir bilgisayarda ölçülmüş/hesaplanmış şu değerleri verdi:

Karmaşıklık	Örnek	$n = 10$	$n = 10^8$
$\log n$	Binary search	0.003 µs	0.027 µs
n	En büyük elemanı bulma, arama	0.01 µs	0.1 s
$n \log n$	Merge sort	0.033 µs	2.66 s
n^2	Selection sort	0.1 µs	115.7 gün
2^n		1 µs	($n = 100$ için) 4×10^{13} yıl
$n!$		3.63 ms	

- n küçükken fark önemsizdir. Gerçek problemlerde n büyüktür: bir bankanın milyonlarca müşterisi, üniversitenin 50-60 bin öğrencisi vardır.
- Aynı işi yapan iki algoritmadan zamanı iyi kullanan seçilmeli.
- Yer verimliliği: Sıralama algoritmalarında "yedek dizi kullanmayın" denmesinin sebebi budur. Milyonlarca eleman için ikinci bir dizi ciddi yer israfıdır.
- Zaman ve yer verimliliği son yıllarda yeniden önem kazandı:
  - Gömülü sistemler (embedded systems), örneğin akıllı telefonlar.
  - Derin öğrenme (deep learning) modelleri: milyonlarca veri, aylarca süren eğitimler.

Çalışma Süresini Etkileyen Faktörler

- **Donanım:** İşlemci hızı, bellek miktarı.
- **Derleyici:** Kod üretimi (code generation) sırasında makine kodunun ne kadar verimli üretildiği.
- **Programlama dili:** Aynı algoritma C, Java ve Assembly'de farklı hızda çalışır.
- **Veri:**
  - Eleman sayısı (n).
  - Başlangıç sıralaması (initial ordering): sıralıya yakın bir diziyi sıralamak, rastgele bir diziyi sıralamaktan farklı sürer.
- İlk üç faktör sisteme bağlıdır. Algoritmanın kendisi ise sistemden bağımsızdır. Bu yüzden algoritma çalıştırılıp süresi ölçülerek değil, tek başına analiz edilerek değerlendirilir.

Adım Sayarak Karmaşıklık Hesabı

Tahtadaki örnek:

```c
int count = 0, i, n;
for (i = 0; i < n; i++)
{
    if (a[i] == 0)
        count++;
}
```

İşlem	Sıklık
Değişken tanımlama (variable declaration)	3
Atama (assignment)	2
`i < n` karşılaştırması	$n+1$
`a[i] == 0` karşılaştırması	n
Dizi erişimi (array access)	n
Artırma (increment)	n ile $2n$ arası

- `i < n` kontrolü $n+1$ kez yapılır. $(n+1)$'inci kontrolde $i = n$ olduğu anlaşılır ve döngüden çıkılır.
- `a[i] == 0` döngünün içinde olduğu için n kez yapılır.
- Artırma: `i++` her zaman n kez yapılır. `count++` dizideki sıfır sayısı kadar yapılır: hiç sıfır yoksa 0, dizi tamamen sıfırsa n kez. Toplam n ile $2n$ arasıdır.
- En kötü durumda toplam yaklaşık $5n + 6$ adım çıkar.
- Bir öğrencinin yazdığı algoritma $4n + 6$, diğerininki $5n + 12$ çıkabilir. Bu farkların önemi yoktur. Hepsi $O(n)$'dir.

Temel İşlem ve Sadeleştirme

- Komut komut saymak gereksizdir. Sabit sayıdaki işlemin n büyüdüğünde etkisi yoktur.
- **Temel işlem (basic operation):** n'e bağlı olarak en çok tekrar eden iştir. Karmaşıklık bu işe göre hesaplanır.
- Bir algoritmada n kez dönen bir döngü ve ardından n^2 kez dönen bir döngü varsa karmaşıklık n^2'ye göre belirlenir.
- $4n^2 + 2n + 9$ için:
  1. Sabitten kurtul: $4n^2 + 2n$.
  2. Katsayılardan kurtul: $4n^2$ ile $5n^2$ arasında ciddi fark yoktur.
  3. En önemli bileşeni al: n^2.
- Bu sadeleştirme $n \to \infty$ varsayımıyla geçerlidir. n^2 algoritması 115.7 gün sürüyorsa $n^2 + n$ algoritması 115.7 gün + 0.1 s sürer; fark önemsizdir.

Asimptotik Notasyon

- Donanım, derleyici ve dilden bağımsız, $n \to \infty$ için yapılan analize asimptotik analiz denir.
- Sonucu göstermek için asimptotik notasyon kullanılır:
  - **Big O:** asimptotik üst sınır.
  - **Big Theta (Θ):** sıkı sınır.
  - **Big Omega (Ω):** alt sınır.
- $4n^2 + 5n$ için $O(n^2)$ yazılır. Üst sınır olduğu için $O(n^3)$ veya $O(n^4)$ yazmak da teknik olarak doğrudur. Ama kimse algoritmasını olduğundan daha karmaşık göstermek istemez; anlamlı olan $O(n^2)$'dir.
- **İç içe iki döngü, m satır ve n sütun:**
  - m sınırlı ve küçükse (örneğin en fazla 100) $O(n)$ denebilir.
  - m de n gibi büyüyorsa $O(n^2)$ veya kullanım yerine göre $O(m \cdot n)$ denmelidir.
  - Karar, algoritmanın ne yaptığına bakılarak verilir.

Best, Worst ve Average Case

- **Dizinin en büyük elemanı:** Diziye sonuna kadar bakmak zorunludur. Best ve worst case aynıdır: n adım.

- **Sırasız dizide eleman arama:** Elemanın yerine göre adım sayısı değişir. Örneğin `2, 1, 4, 7, 9, 4` dizisinde 2 aranıyorsa 1 adım, 9 aranıyorsa 5 adım.
- Best/worst/average farkı adım sayısı belli olmayan algoritmalarda anlam kazanır. Bunlar genelde `while` ile yazılan, bir koşul sağlanınca çıkılan algoritmalardır.

Doğrusal Arama

```
int search(int a[], int n, int x)
{
    int i = 0;
    while (i < n && a[i] != x)
        i++;
    if (i < n)
        return i;
    else
        return -1;
}
```

- Döngüden iki sebeple çıkılır: $i = n$ olmuştur (bulunamadı) veya `a[i] == x` olmuştur (bulundu).
- Hocanın izlemesi, dizi `3, 5, 1, 2, 7`, $n = 5$:
  - $x = 1$: $i = 0$ (3), $i = 1$ (5), $i = 2$ (1) bulundu, 2 döner.
  - $x = 4$: Tüm elemanlara bakılır, $i = 5$ olur, -1 döner.
- Temel işlem: döngüdeki karşılaştırma.
- **Worst case:** Eleman dizide yoksa veya son gözdeyse n karşılaştırma yapılır, $O(n)$.
- **Best case:** Eleman ilk gözdeyse 1 karşılaştırma yapılır, $O(1)$.
- **Average case (beklenen adım sayısı, expected steps for random input):**
  - Elemanın dizide bulunma olasılığı p olsun.
  - Rastgele durumda her pozisyon eşit olasılıklıdır. Elemanın i'inci pozisyonda olma olasılığı p/n, bu durumda yapılan karşılaştırma sayısı i'dir.
  - Eleman yoksa (olasılık $1 - p$) n karşılaştırma yapılır.

$$C_{avg}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}\right] + n(1-p) = \frac{p}{n} \cdot \frac{n(n+1)}{2} + n(1-p) = \frac{p(n+1)}{2} + n(1-p)$$

- Başarılı arama (successful search), $p = 1$: $C_{avg} = \frac{n+1}{2}$.
- Başarısız arama (unsuccessful search), $p = 0$: $C_{avg} = n$, bu zaten worst case'dir.
- Ortalama durum $\frac{n+1}{2}$'dir. Sabitler atılınca $O(n)$ olur.
- Bir öğrenci $C(n)$'deki C'nin anlamını sordu. Hoca karşılaştırma (comparison) veya karmaşıklık için kullandığını, $T(n)$ de yazılabileceğini söyledi. Önemli olan sonucu Big O ile vermektir.

Hangisi Önemli?

- Best case şansa bağlıdır, önemi yoktur.
- Normalde worst case esas alınır.
- Kaynak yetersizse veya maliyet önemliyse average case'e göre plan yapılır:
  - GSM operatörleri altyapıyı ortalama trafiğe göre kurar. Depremde herkes aynı anda aradığında sistem çöker.
  - ÖSYM sınav sonuçları açıklandığında site çöker.
  - Bayram sabahı aramalarda yavaşlık olur.
- Quicksort: average $O(n \log n)$, worst $O(n^2)$. Buna rağmen en çok kullanılan sıralama algoritmasıdır, çünkü pratikte genelde average case oluşur. Merge sort ve heap sort da $O(n \log n)$'dir.

Insertion Sort

Hocanın izlemesi, dizi `3, 5, 2, 1, 4, 7, 8`:

Adım	Yerleştirilen	Dizi
Başlangıç		3, 5, 2, 1, 4, 7, 8
1	5 (3'ten büyük, yerinde)	3, 5, 2, 1, 4, 7, 8
2	2	2, 3, 5, 1, 4, 7, 8
3	1	1, 2, 3, 5, 4, 7, 8
4	4	1, 2, 3, 4, 5, 7, 8
5	7 (5'ten büyük, yerinde)	1, 2, 3, 4, 5, 7, 8
6	8 (7'den büyük, yerinde)	1, 2, 3, 4, 5, 7, 8

- Avantajı: Baş taraf her zaman sıralıdır. Yerleştirilen eleman solundakinden büyükse daha geriye bakmaya gerek yoktur. Selection sort ve bubble sort'ta böyle bir avantaj yoktur.
- **Verimlilik ipuçları:**
  - Yerleştirilecek elemanı geçici bir değişkene (`tmp`) alın. Her karşılaştırmada `A[i]`'ye tekrar tekrar erişmek yavaştır.
  - Sürekli yer değiştirmek (swap) yerine büyük elemanları sağa kaydırın, elemanı en sonda doğru yere koyun.
  - Hoca `i = i + 1` yerine `i++` kullanılmasını, `i++`'nın düşük seviyede tek bir increment komutuna karşılık geldiğini söyledi (bkz. ders sonrası notlar).

Tahtadaki akış diyagramının koda karşılığı:

```
for (i = 1; i <= n - 1; i++)
{
    tmp = A[i];
    j = i;
    while (j > 0 && tmp < A[j - 1])
    {
        A[j] = A[j - 1];
        j = j - 1;
    }
    A[j] = tmp;
}
```

- `3, 5, 2, ...` dizisinde $i = 2$ için: `tmp = 2`. $j = 2$: $2 < 5$, 5 sağa kayar. $j = 1$: $2 < 3$, 3 sağa kayar. $j = 0$: döngü biter, `A[0] = 2`. Dizi `2, 3, 5, ...` olur.
- Temel işlem: iç döngüdeki karşılaştırma.
- **Worst case:** Dizi büyükten küçüğe sıralı. i'inci turda i karşılaştırma yapılır.

$$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} \Rightarrow O(n^2)$$

- **Best case:** Dizi zaten küçükten büyüğe sıralı. Her turda 1 karşılaştırma yapılır, toplam $n - 1$, yani $O(n)$.
- Sıralıya yakın dizilerde (birkaç yeni eleman sona eklenmiş gibi) karmaşıklık n'e yakındır.
- Küçük dizilerde $O(n \log n)$ algoritmalar yerine insertion sort tercih edilir. Hoca, Java'nın küçük diziler için insertion sort kullandığını, quicksort'un küçük parçalarda da insertion sort'tan yararlanılabildiğini söyledi.
- Sebebi: Quicksort ve merge sort özyinelemelidir (recursive). Sürekli alt program çağırmanın kendi yükü vardır.
- Merge sort her zaman $O(n \log n)$'dir ama yedek dizi kullanır. Selection sort her durumda $O(n^2)$'dir; bu yüzden tercih edilmez. $O(n^2)$ algoritmalar içinde en çok tercih edilen insertion sort'tur.

Karmaşıklık Sınıfları

Notasyon	Ad	Örnek
$O(1)$	Sabit (constant)	İki sayıdan küçüğünü bulma, artık yıl kontrolü, hashing

Notasyon	Ad	Örnek
$O(\log n)$	Logaritmik	Binary search; her adımda problem ikiye bölünür (1024 eleman 10 adımda 1'e iner)
$O(n)$	Doğrusal (linear)	En büyük elemanı bulma
$O(n \log n)$	Linearitmik (linearithmic)	Quicksort, merge sort
$O(n^2)$	Karesel (quadratic)	Selection sort, bubble sort; iç içe iki döngü
$O(2^n)$	Üstel (exponential)	

- $O(1)$ "tek karşılaştırma" demek değildir. 10 veya 50 karşılaştırma da olabilir; önemli olan n'e bağlı olmamasıdır.

Kullanışlı Toplam Formülleri

- $\sum_{i=1}^{n} 1 = n$
- $\sum_{i=l}^{u} 1 = u - l + 1$ ($l \leq u$ için)
- $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$
- $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3}$
- $\sum_{i=l}^{u} c \cdot a_i = c \sum_{i=l}^{u} a_i$
- $\sum_{i=1}^{n} \log i \approx n \log n$

Ders sonrası notlar

- Hoca ilk tabloda değerleri önce milisaniye diye okudu, sonra mikrosaniye olarak düzeltti. Doğrusu mikrosaniyedir (µs); yalnız $n!$ için değer 3.63 ms'dir.
- Hoca $n = 10$ için $\log n$ değerini "0.03" diye okudu. Doğru değer yaklaşık 0.003 µs'dir ($\lg 10 \approx 3.3$ işlem × 1 ns).
- Altyazıda n^2 için 100 milyon elemanda "%10.5.7 gün" geçiyor. Doğru değer yaklaşık 115.7 gündür (10^{16} ns).
- Hoca $n \log n$ algoritmalarını "linear big time" diye adlandırdı. Yerleşik terim "linearithmic" veya "n log n time"dır.
- Hoca derste `count+++` yazdığını fark edip öğrencilerin uyarmamasına takıldı. Doğrusu `count++`'dır.
- Hoca Java'nın sıralamada quicksort kullandığını söyledi. Bu altyazıdan doğrulanamaz; büyük olasılıkla Java ilkel tiplerde (int, double) Dual-Pivot Quicksort, nesnelerde ise TimSort (merge sort + insertion sort) kullanır. Küçük parçalarda insertion sort kullanımı hocanın söylediğiyle uyumludur.
- `i++` ile `i = i + 1` arasındaki hız farkı günümüz derleyicilerinde genelde yoktur. Bu makinedeki gcc ile iki biçim `-O0` (optimizasyonsuz) ile de derlendi; aynı makine kodu üretildi, yalnız etiket adları farklıydı. Hocanın dediği büyük olasılıkla eski veya basit derleyiciler için geçerlidir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.